# BERR2243
# DATABASE AND CLOUD SYSTEMBERN
## SEMESTER 2 2025/2026

## SMART GREENHOUSE MONITORING SYSTEM

## LECTURE NAME: PROFESOR MADYA DR. SOO YEW GUAN

| NO | NAME | MATRIX NUMBER | PHOTO |
|---|---|---|---|
| 1. | WAN MARYAM NAZIHAH BINTI WAN MOHD ZUKRI | B122510466 |  |
| 2. | NUR AFIQAH NAJWA BINTI MOHD RIZAL | B122510466 |  |

| | | | |
|---|---|---|---|
| **3.** | **NUR INSYIRAH BINTI REDZUAN** | **B122510479** |  |
| **4.** | **YALINI A/P KARTHEGES** | **B122510486** |  |

# TABLE OF CONTENT

## 1.0 Introduction

Efficient greenhouse systems constitute one of the vital applications of current precision agriculture technologies, whereby there is continuous monitoring of the environment parameters like temperatures, humidity, and soil parameters to enhance the efficiency of plant development and resource use. As sustainable farming methods gain momentum, there has been a need for real-time decision-making by farmers and researchers through the use of monitoring systems that rely on data.

The name of his project is Smart Greenhouse Monitoring System and is developed as cloud-based data management system emphasizing the data collection, storage, and monitoring with a NoSQL database system. Despite the usual IoT design that requires hardware data acquisition through microcontrollers and sensors, this design has implemented the IoT design principles in a software environment. The hardware part has been replaced by Postman where JSON packets are created and sent to the MongoDB database server through HTTP requests simulating the data collected from greenhouse sensors. The packets are then stored in MongoDB database that acts as the database of this design. This implementation reflects the end-to-end cloud data pipeline from raw environmental data to structured NoSQL document storage.

In addition, this system shows the fundamentals of cloud database integration such as real-time data insertion, schema creation for the sensor telemetry, and data security via database authentication methods. Data obtained by the sensors can be used to monitor the state inside the greenhouse as well as further dashboard development. This paper is going to discuss the architecture of the system, implementation using MongoDB and Postman, and difficulties experienced during the development process, especially data structure creation and its transfer to the database.

## 2.0 System Architecture

The Smart Greenhouse Monitoring System utilizes cloud architecture for the simulation, processing, storage, and visualization of environmental data from the greenhouse. In place of using real-time IoT devices like ESP32, the Smart Greenhouse Monitoring System uses the Postman tool to simulate the data that the sensors will provide, through sending HTTP requests with JSON format of the data from the greenhouse.

The simulated data that is generated using Postman is then consumed by the middleware, known as Node-RED, which takes care of processing the incoming requests and controlling the flow of data. Node-RED checks the JSON payload that is being received and forwards the data to the MongoDB Atlas for storage purposes. MongoDB Atlas offers a scalable and secure cloud database service for storing structured and unstructured data of greenhouses.

Lastly, the data is fetched from MongoDB Atlas by MongoDB Charts and displayed using visualization techniques, thus making it possible for one to keep track of factors like temperature, humidity, moisture, and light intensity within the greenhouse in real-time. This is how cloud computing and database management software can be used to develop an efficient greenhouse monitoring system.

Components of the System Architecture

- Postman – Emits fake data from the sensors in the greenhouse by sending HTTP POST messages in JSON format.
- Node-RED – Processes the incoming messages and sends them to the database.
- MongoDB Atlas – Stores the data from the sensors in the greenhouse in the form of NoSQL documents.
- MongoDB Charts Dashboard – Represents the data through charts on the dashboard.

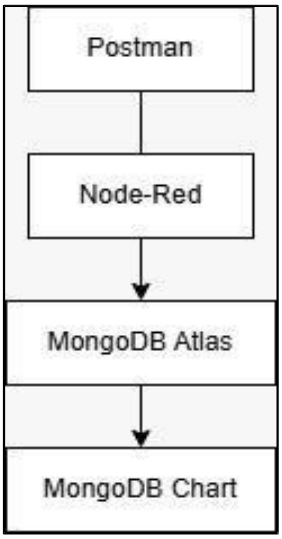


**Figure 1**: System Architecture

## 3.0 System Implementation

### 3.1 Telemetry Data Flow

The Smart Greenhouse Monitoring System starts with the use of Postman for simulating the sensor readings by making HTTP POST request to Node-RED in JSON format. The data sent contains environmental readings and device identification data for authentication.

Once the request has been received, a series of functions takes place in order to process the request. These two functions include "Save Original Data," which stores the original request, and "Prepare Telemetry Data," which extracts the necessary data fields from the original request and prepares it for storage in the database. Afterwards, the processed data goes through validation before being stored in the MongoDB Atlas.

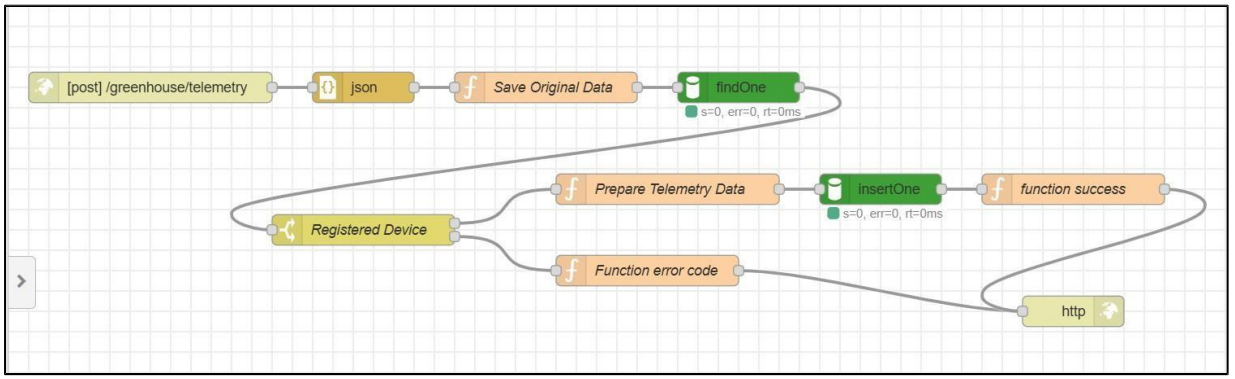


**Figure 2**: Telemetry to MongoDB Flow

### 3.2 Save Original Data Function

Save Original Data node is the first node in the Node-RED processing chain. The role of this node is to save the original JSON payload received from Postman without any modification. It helps the system save the original telemetry data for future reference and troubleshooting.

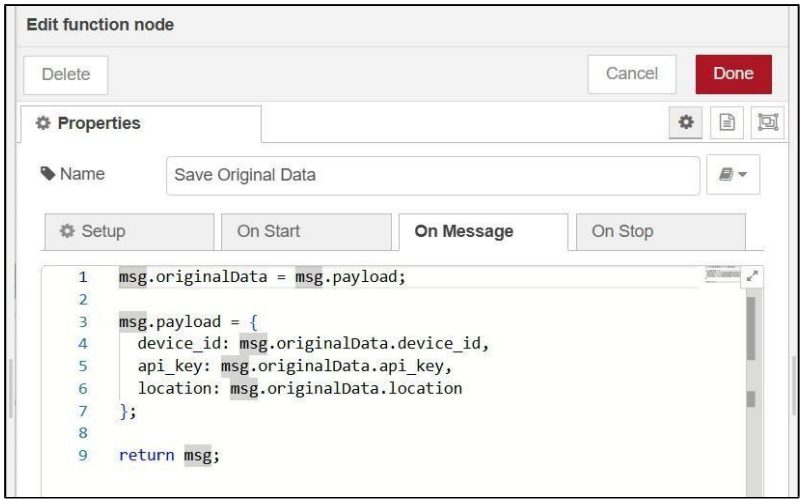


**Figure 3**: Save Original Data' Function Setup

### 3.3 Data Validation

In order to maintain data integrity and security of the system, Node-RED validates each incoming request before saving it to the database. This validation checks the device data submitted against the metadata saved in the Metadata collection.

Three validation criteria are implemented:

- Device ID verification
- Location verification
- API key authentication

When these three criteria are fulfilled, the system sends the telemetry information to the Telemetry collection and generates a success response. When one of these criteria is not met, the request is denied with the relevant error message.

### 3.4 Prepare Telemetry Data Function

After storing the payload, the Prepare Telemetry Data function parses the required fields from the JSON request and reformats the data to be stored in MongoDB Atlas. This function ensures that there is consistency in the structure of the document to facilitate validation and database insertion.

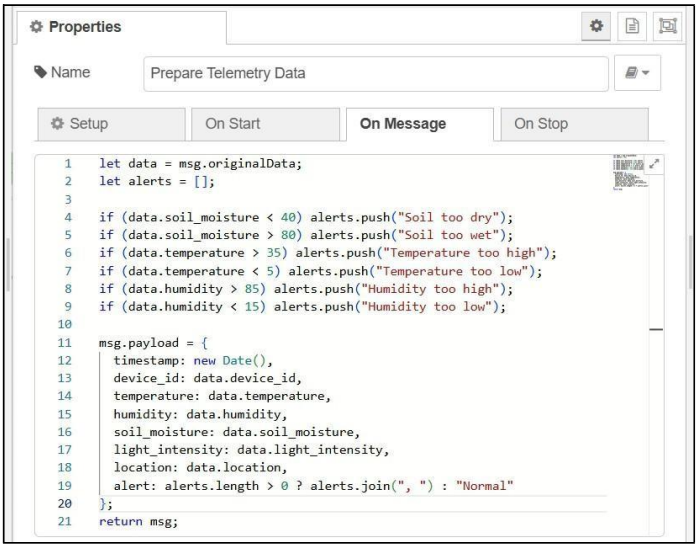


**Figure 4**: Prepare Telemetry Data' Function Setup

### 3.5 Database Design

MongoDB Atlas stores greenhouse monitoring data using two separate collections.

- *Metadata Collection*

The Metadata collection stores information about authorised greenhouse devices, including their device ID, location, and API key. This collection acts as the reference database during validation.

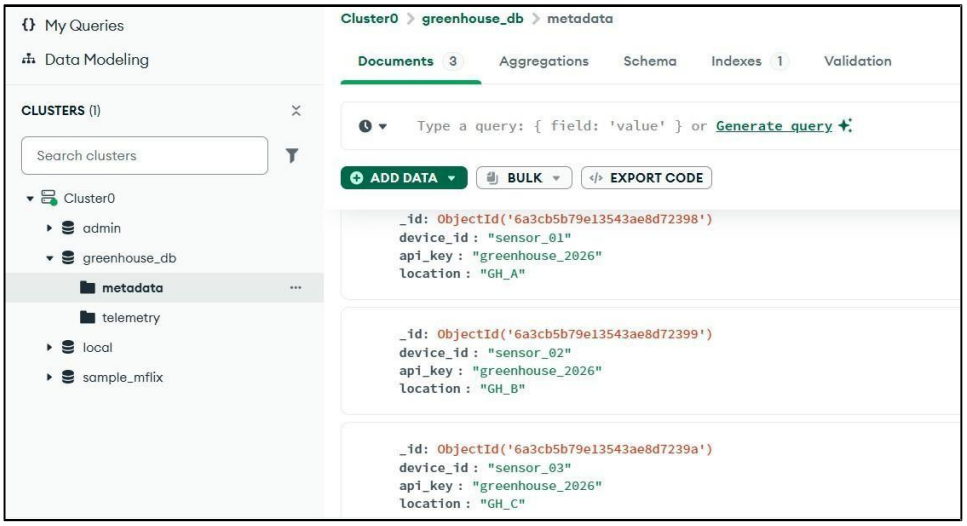


**Figure 3**: Registered Device in Metadata Collection

- *Telemetry Collection*

The Telemetry collection stores all validated environmental readings received from Node-RED. Each document contains greenhouse sensor readings together with the associated timestamp and device information.

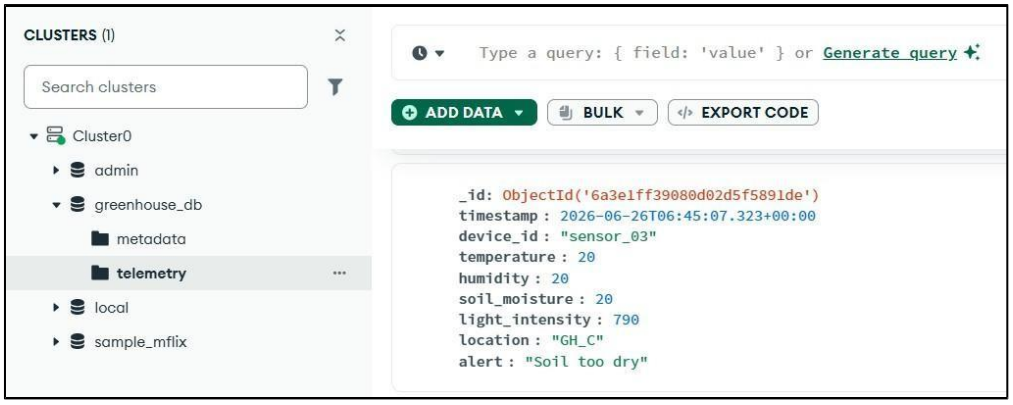


**Figure 4**: Stored Data in Telemetry Collection

### 3.6 Success and Error Handling

Function Success node produces a successful output when the telemetry data has undergone all validation tests and has been successfully saved into MongoDB Atlas. This function ensures that the request has been successfully processed and that the client is notified about successful data saving.


**Figure 5**: Function Success' Function Setup

The Function Error Code node responds to unsuccessful requests by providing suitable error messages when the validation process does not succeed. This function increases the reliability of the system by ensuring that only valid telemetry data is saved in the database.

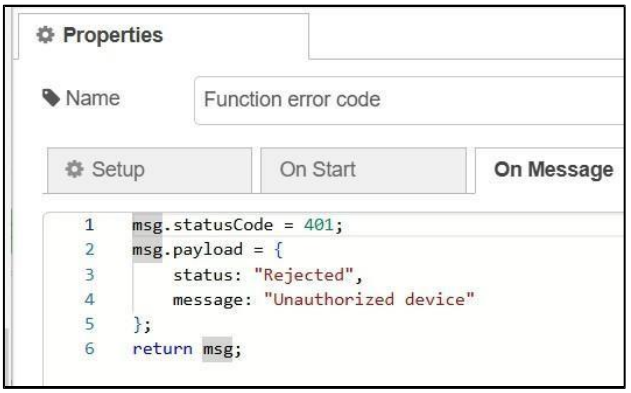


**Figure 6**: Function Error Code' Function Setup

The system incorporates a number of Node-RED functions that help make it reliable and offer feedback to the users. Upon receiving valid telemetry data, the function success node gives out an output that indicates the successful storage of the data in MongoDB Atlas. If invalid information is detected, dedicated error functions return appropriate responses, including:

- Invalid Device ID
- Incorrect Device Location
- Invalid API Key

These validation mechanisms prevent unauthorised or incorrect data from being stored in the database.

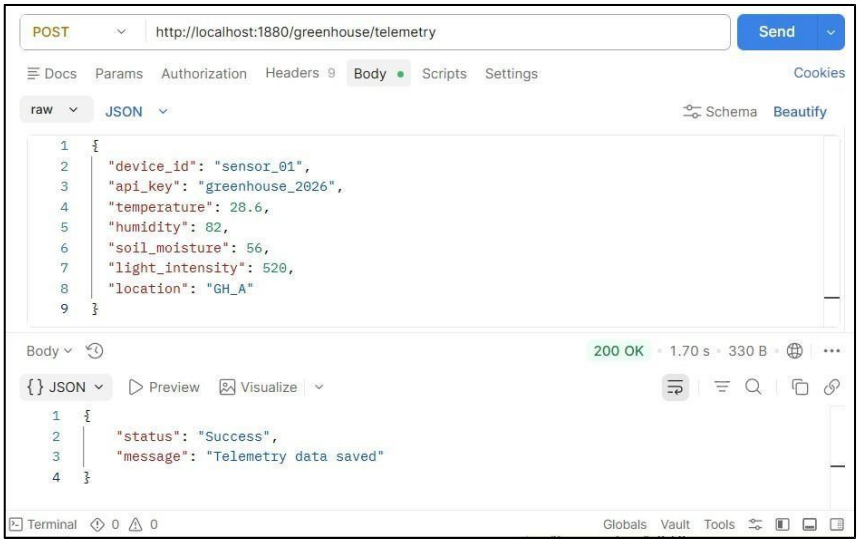


**Figure 7**: Successful Body Sent

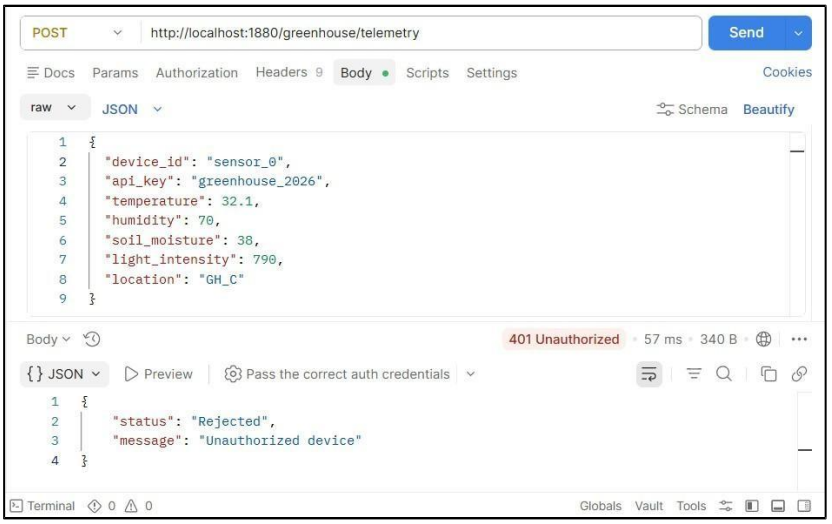


**Figure 8**: Wrong Device ID

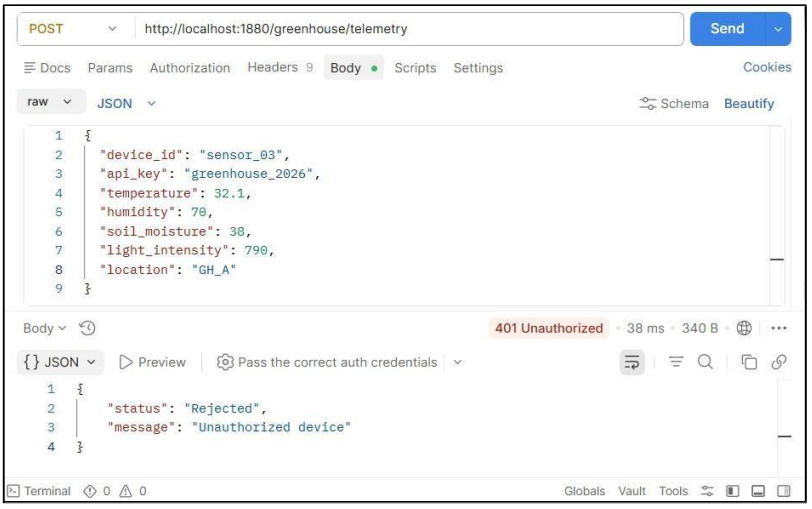


**Figure 9**: Wrong Location on Wrong Device ID

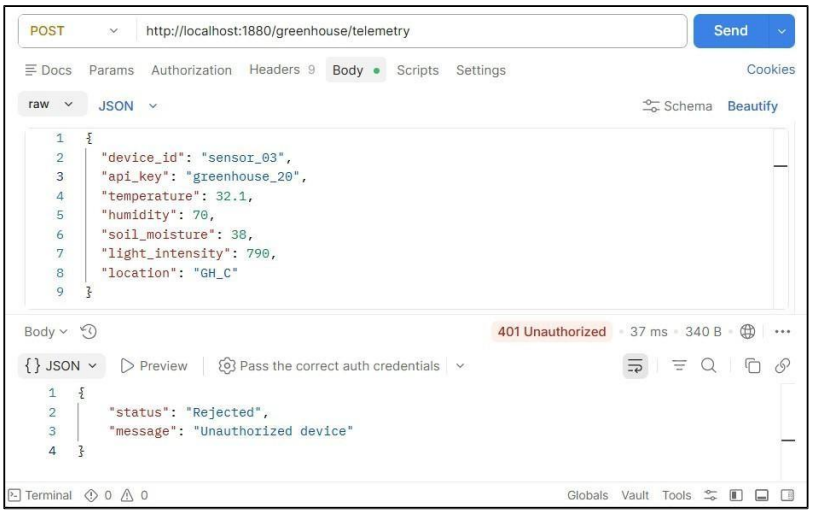


**Figure 10**: Wrong API Key

### 3.7 Alert Function

The other feature incorporated in the Smart Greenhouse Monitoring System is the alert functionality provided by Node-RED. The functionality involves the continuous assessment of incoming telemetry data based on specified criteria. When the system detects any abnormalities in the readings from the greenhouse, for example, high temperatures or low moisture content in the soil, it sends out an alert.

```
if (data.soil_moisture < 40) alerts.push("Soil too dry");
if (data.soil_moisture > 80) alerts.push("Soil too wet");
if (data.temperature > 35) alerts.push("Temperature too high");
if (data.temperature < 5) alerts.push("Temperature too low");
if (data.humidity > 85) alerts.push("Humidity too high");
if (data.humidity < 15) alerts.push("Humidity too low");
```

**Figure 11**: Alert Setup in Node-RED Function

## 4.0 Challenges Encountered

Technical problems faced during the implementation of the Smart Greenhouse Monitoring System include:

- One of the biggest technical problems was related to setting up the configuration of the Node-RED data flow to ensure correct processing and validation of telemetry data prior to adding them to the MongoDB Atlas. Communication between Postman, Node-RED, and MongoDB has been done through repeated testing and debugging.
- Secondly, there have been some problems related to the implementation of the validation for device ID, location, and API key. This has sometimes caused failed authentication and database insertion.

## 5.0 Security Implementation

Data validation and database authentication were used as the basis for security in the system. All requests that come in must be authenticated based on the registered device information found in the Metadata collection. No request with invalid device IDs, device location, and API key will be allowed to save their telemetry data in the MongoDB Atlas.

Further security for the MongoDB Atlas involves using authenticated users and network access control. This will ensure the protection of integrity of the greenhouse monitoring data.

## 6.0 Conclusion

To conclude, the Smart Greenhouse Monitoring System presents the effective deployment of a cloud-based monitoring solution to collect, process, store, and visualize environmental data in greenhouses. The use of Postman, Node-RED, MongoDB Atlas, and MongoDB Charts creates an efficient workflow from collecting data to storing it in the cloud using NoSQL databases that imitate the process of data collection without IoT devices in a realistic manner. This enables the system to collect telemetry data in JSON format, validate devices' data, store it in the cloud-based database, and visualize it in a dashboard.

During the development process, there was great practical experience gained in cloud databases, RESTful API, middleware, and NoSQL databases. Data validation based on device ID, location, and API key verification made the project more reliable and secure as it guaranteed that the telemetry data entered in the database is authorized. Moreover, the data processing procedure was made simpler due to the use of Node-RED, and the choice of MongoDB Atlas as a platform for data management became quite logical.

Although there have been some difficulties faced in the process of integration and validation of the data, these problems were eventually sorted out by way of testing, debugging, and optimizing the Node-RED flow. The entire project has been successful and has met its targets and has proved how cloud technology can be utilized in designing a smart monitoring system. Future developments in the system could include the use of real IoT sensors, automation of greenhouse operations, and predictive analysis.